

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Департамент программной инженерии

СОГЛАСОВАНО

Научный руководитель,
профессор департамента программной
инженерии ФКН НИУ ВШЭ

_____ В. В. Шилов
«_____» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»

_____ Н. А. Павлочев
«_____» _____ 2026 г.

Выпускная квалификационная работа

на тему: **Генератор данных для аналитического бенчмарка СУБД ТРС-Н**

по направлению подготовки 09.03.04 «Программная инженерия»

СОГЛАСОВАНО

Консультант,
разработчик Яндекс Технологии

_____ М. Т. Садуллаев
«_____» _____ 2026 г.

ВЫПОЛНИЛ

студент группы БПИ225
образовательной программы
09.03.04 «Программная инженерия»

_____ М. А. Дергоусов
«_____» _____ 2026 г.

Реферат

Выпускная квалификационная работа посвящена разработке генератора данных для аналитического бенчмарка СУБД TPC-H. Актуальность работы определяется тем, что традиционные средства формирования набора TPC-H ориентированы преимущественно на вывод строковых файлов и не учитывают требований современной инфраструктуры аналитических систем, использующей колоночное представление данных, удаленные объектные хранилища и встроенные сценарии генерации данных в оперативной памяти.

Целью работы является разработка генератора Schengen, обеспечивающего детерминированное формирование таблиц TPC-H с поддержкой параллельной генерации, прямой записи в современные форматы хранения и отделением модуля генерации от слоя сериализации.

Результатом работы является программная система на языке C++, выполняющая генерацию таблиц TPC-H, формирование батчей Apache Arrow [1], запись в форматы Parquet [2], ORC [3], Lance [4] и Vortex [5], а также выгрузку результатов на локальные и S3-совместимые хранилища. Отдельно реализована возможность использования генератора как встраиваемого источника данных без промежуточной записи на диск, что подтверждено интеграцией с СУБД DuckDB [6].

Данная работа состоит из 32 страниц, 3 глав, 3 рисунков, 19 листингов, 3 таблиц, 8 приложений. Использовано 14 источников.

Ключевые слова: OLAP; СУБД; TPC-H; Apache Arrow; Parquet; ORC; DuckDB.

Abstract

This thesis is devoted to the development of a data generator for the TPC-H analytical DBMS benchmark. The relevance of the work is determined by the fact that traditional TPC-H data generation tools are mostly focused on row-oriented text output and do not meet the requirements of modern analytical infrastructures that rely on columnar data representation, remote object storage, and in-memory data generation scenarios.

The goal of the work is to develop the Schengen generator that provides deterministic TPC-H table generation, parallel execution, direct output to modern storage formats, and a clear separation between the data generation core and the serialization layer.

The result of the work is a C++ software system that generates TPC-H tables, builds Apache Arrow [1] batches, writes them to Parquet [2], ORC [3], Lance [4], and Vortex [5] formats, and stores the output either on a local file system or in S3-compatible storage. The generator can also be embedded directly into analytical systems without intermediate disk materialization, which is demonstrated by its integration with DuckDB [6].

The paper contains 32 pages, 3 chapters, 3 figures, 19 listings, 3 tables, 8 appendices. 14 sources are used.

Keywords: OLAP; DBMS; TPC-H; Apache Arrow; Parquet; ORC; DuckDB.

Содержание

Реферат	2
Abstract	3
Основные определения, термины и сокращения	6
Введение	7
Глава 1 Предметная область и анализ аналогов	9
1.1 ТРС-Н как стандарт аналитического бенчмаркинга	9
1.1.1 Назначение бенчмарка	9
1.1.2 Состав набора данных	9
1.1.3 Актуальность ТРС-Н в современных условиях	9
1.2 Требования современной аналитической инфраструктуры к генератору данных	10
1.2.1 Требования к выходному представлению данных	10
1.2.2 Требования к размещению и доступу к данным	10
1.2.3 Архитектурные требования	10
1.3 Анализ существующих решений	11
1.3.1 Генератор dbgen	11
1.3.1.1 Достоинства	11
1.3.1.2 Ограничения	11
1.3.2 Генератор tpchgen-rs	11
1.3.2.1 Достоинства	11
1.3.2.2 Ограничения	11
1.3.3 Сравнительный анализ и обоснование направления разработки . .	11
Выводы по главе	12
Глава 2 Архитектура генератора данных	13
2.1 Архитектурные требования к масштабируемой генерации	13
2.2 Общая идея алгоритма	13
2.3 Параллельная генерация на основе перемотки линейного генератора . . .	14
2.3.1 Математическая модель генератора	14
2.3.2 Алгоритм быстрой перемотки и асимптотика	15
2.4 Архитектура слоя сериализации	15
2.4.1 Форматно-зависимые компоненты записи	15
2.4.2 Линейный и детерминированный режим записи	16
2.5 Разделение модулей генерации и сериализации	16
Выводы по главе	17

Глава 3 Реализация генератора данных	18
3.1 Реализация генерации таблиц	18
3.1.1 Организация генерации по партициям	18
3.1.2 Генерация значений и использование текстового пула	18
3.1.3 Формирование таблиц набора данных	21
3.1.3.1 Таблица region	21
3.1.3.2 Таблица nation	21
3.1.3.3 Таблица supplier	22
3.1.3.4 Таблица part	22
3.1.3.5 Таблица partsupp	23
3.1.3.6 Таблица customer	23
3.1.3.7 Таблица orders	24
3.1.3.8 Таблица lineitem	25
3.2 Реализация слоя сериализации	25
3.2.1 Общий интерфейс записи	25
3.2.2 Форматы с управляемым стримингом	26
3.2.3 Форматы с нативным стримингом	27
3.3 Интерфейс запуска и сценарии эксплуатации	27
3.3.1 Параметры командной строки	27
3.3.2 Типовые сценарии запуска	28
3.4 Реализация встраиваемого сценария	29
3.4.1 Идея встроенного сценария	29
3.4.2 Демонстрационный прототип для DuckDB	29
Выводы по главе	30
Заключение	31
Список использованных источников	32
Приложение А	33
Приложение Б	34
Приложение В	35
Приложение Г	36
Приложение Д	37
Приложение Е	38
Приложение Ж	39
Приложение З	41

Основные определения, термины и сокращения

Термин	Определение
Apache Arrow	Открытая платформа для представления и обмена колоночными данными в оперативной памяти [1].
DuckDB	Встраиваемая аналитическая СУБД, ориентированная на обработку данных в колоночном представлении [7].
Lance	Колоночный формат и формат набора данных, ориентированный на аналитическую обработку и быструю выборку [4].
ORC	Колоночный формат хранения данных, применяемый в аналитических системах и ориентированный на эффективное сжатие и сканирование [3].
Parquet	Колоночный формат хранения данных, предназначенный для аналитической обработки и межсистемного обмена [2].
Scale factor	Параметр спецификации TPC-H, определяющий размер формируемого набора данных в гигабайтах [8].
TPC-H	Стандартный аналитический бенчмарк, включающий фиксированную схему таблиц, правила генерации данных и набор запросов для оценки производительности СУБД [8].
Vortex	Адаптивный колоночный формат данных для систем, использующих Apache Arrow [5].
Lakehouse-система	Класс аналитических систем, объединяющих хранение данных в файловом озере с табличными слоями метаданных; примером такого слоя является Apache Iceberg [9].
OLAP	Класс систем и методов аналитической обработки данных, ориентированных на выполнение сложных запросов над большими массивами информации.
Amazon S3	Объектное хранилище и программный интерфейс доступа к данным, де-факто используемые как стандарт для совместимых облачных хранилищ.
Батч	Группа строк, объединенная для совместной обработки, передачи или сериализации.
Партиция	Логическая часть таблицы, генерируемая и обрабатываемая как самостоятельный фрагмент общего набора данных.
Сериализация	Преобразование внутреннего представления данных в формат, пригодный для записи на носитель или передачи между компонентами.
СУБД	Система управления базами данных, обеспечивающая хранение, обработку и доступ к данным.

Введение

Актуальность темы

Аналитические СУБД и системы поддержки принятия решений традиционно оцениваются с помощью воспроизводимых бенчмарков, задающих как структуру данных, так и характер рабочей нагрузки. Одним из наиболее распространенных стандартов такого типа является ТРС-Н, определяющий фиксированный набор таблиц, правила генерации данных и набор аналитических запросов для сравнения систем между собой [8]. Значимость воспроизводимых генераторов данных в этой области связана не только с корректностью самих измерений, но и с возможностью масштабирования экспериментов на многопроцессорных и распределенных платформах [10].

Классические средства подготовки набора ТРС-Н исторически ориентированы на вывод строковых файлов формата `tbl`, которые затем требуется преобразовывать в целевые форматы хранения или загружать в СУБД отдельным конвейером постобработки [11]. Такой подход плохо согласуется с современной инфраструктурой аналитической обработки данных, в которой широко используются внутрипамятные колоночные представления Apache Arrow [1], колоночные форматы Parquet [2], ORC [3], Lance [4] и Vortex [5], а также современные lakehouse-системы и их табличные слои, например Apache Iceberg [9].

В этих условиях возникает потребность в генераторе, который сочетает детерминированность эталонного ТРС-Н [8], параллельную генерацию, прямую запись в современные колоночные форматы и возможность выгрузки данных не только на локальный диск, но и в S3-совместимые объектные хранилища. Дополнительное значение имеет отделение модуля генерации от слоя сериализации, поскольку такая архитектура позволяет использовать генератор не только как автономную утилиту, но и как встраиваемый источник батчей Apache Arrow [1].

Цель и задачи работы

Целью настоящей выпускной квалификационной работы является разработка генератора данных Schengen [12] для аналитического бенчмарка ТРС-Н [8], обеспечивающего прямое формирование данных в современных форматах хранения и поддерживающего сценарии встроенной генерации в оперативной памяти.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) проанализировать предметную область аналитических бенчмарков и существующие средства генерации данных ТРС-Н [8];
- 2) сформулировать требования к архитектуре генератора, ориентированного на современные аналитические форматы и сценарии хранения;

- 3) разработать модуль детерминированной генерации таблиц ТРС-Н и слой сериализации в поддерживаемые форматы;
- 4) обеспечить возможность прямой интеграции генератора с аналитическими системами без промежуточной записи данных на диск;
- 5) проверить работоспособность выбранных решений на практике, в том числе на примере интеграции с DuckDB [6].

Объект и предмет исследования

Объектом исследования являются программные средства подготовки данных для аналитических бенчмарков.

Предметом исследования являются методы детерминированной генерации таблиц ТРС-Н [8], их представления в колоночной форме и последующей сериализации в современные форматы хранения.

Практическая значимость работы

Практическая значимость работы состоит в том, что разработанный генератор позволяет сократить число промежуточных этапов при подготовке наборов данных для аналитических экспериментов, упростить воспроизведение тестов и расширить сценарии применения генератора за счет работы с локальными и удаленными хранилищами, а также за счет встраивания в OLAP-системы и современные lakehouse-системы.

Структура работы

Структура работы включает введение, главы, посвященные анализу предметной области, проектированию и реализации системы, экспериментальной оценке, заключение и список использованных источников.

Глава 1. Предметная область и анализ аналогов

В данной главе рассматриваются особенности предметной области аналитических бенчмарков, формулируются требования к современному генератору данных ТРС-Н и выполняется сравнительный анализ существующих решений.

1.1. ТРС-Н как стандарт аналитического бенчмаркинга

1.1.1. Назначение бенчмарка

Бенчмарк ТРС-Н занимает устойчивое место в практике оценки аналитических СУБД, поскольку задает одновременно структуру предметной области, правила формирования набора данных и набор запросов, моделирующих типовые сценарии аналитической обработки [8]. В отличие от синтетических тестов, оценивающих отдельные низкоуровневые операции, ТРС-Н ориентирован на проверку сквозной эффективности систем поддержки принятия решений: выполнения агрегирующих запросов, соединений, фильтрации и сортировки над большими таблицами.

1.1.2. Состав набора данных

Схема ТРС-Н включает восемь взаимосвязанных таблиц [8]. К ним относятся `region`, `nation`, `supplier`, `part`, `partsupp`, `customer`, `orders` и `lineitem`. Для каждой таблицы спецификация задает состав атрибутов, правила построения ключей, распределения значений и формулы вычисления зависимых полей. Наличие строго определенных правил генерации делает ТРС-Н удобной основой для воспроизводимых экспериментов: при одинаковом масштабе и корректной реализации генератора результирующий набор данных должен быть эквивалентен эталонному.

1.1.3. Актуальность ТРС-Н в современных условиях

Значение ТРС-Н усиливается тем, что он остается применимым и в современных условиях, когда аналитические системы работают на многоядерных узлах, используют параллельное выполнение и активно опираются на колоночные методы хранения и обработки данных [10]. Соответственно, требования к генератору данных выходят за пределы простой записи строковых файлов и включают поддержку масштабирования, эффективного внутреннего представления и совместимости с современной экосистемой хранения данных.

1.2. Требования современной аналитической инфраструктуры к генератору данных

1.2.1. Требования к выходному представлению данных

Классический сценарий подготовки набора ТРС-Н предполагает формирование строковых файлов, последующую конвертацию, разбиение на партиции и отдельную загрузку в целевую систему [11]. Такой конвейер увеличивает число этапов обработки, требует дополнительного дискового пространства и затрудняет проведение повторяемых экспериментов в случаях, когда необходимо быстро получать наборы данных разного масштаба.

Современная аналитическая инфраструктура предъявляет к генератору данных более широкий набор требований. Прежде всего данные должны формироваться в представлении, согласованном с колоночными моделями обработки, например в виде батчей Apache Arrow [1]. Кроме того, желательна прямая запись в используемые на практике форматы хранения, прежде всего Parquet [2] и ORC [3], а также в новые форматы, ориентированные на интеграцию с аналитическими движками и lakehouse-инфраструктурой, а также на работу с современными lakehouse-системами, использующими табличные слои наподобие Apache Iceberg [9]. К таким форматам относятся Lance [4] и Vortex [5].

1.2.2. Требования к размещению и доступу к данным

Генератор должен учитывать особенности эксплуатации в распределенной инфраструктуре. На практике результаты экспериментов нередко размещаются не только на локальном диске, но и в объектных S3-совместимых хранилищах. Это означает, что генератор должен уметь работать с абстракциями файловых систем и URI, а не предполагать исключительно локальную запись.

1.2.3. Архитектурные требования

Важным архитектурным требованием является отделение собственно генерации данных от слоя сериализации. При таком разделении одна и та же логика формирования таблиц может использоваться как для автономной выгрузки на носитель, так и для встроенных сценариев, в которых данные передаются в оперативной памяти непосредственно потребляющей системе. Этот подход особенно важен для OLAP-движков, где чтение с диска в некоторых экспериментах целесообразно заменять прямой генерацией колоночных батчей.

1.3. Анализ существующих решений

1.3.1. Генератор dbgen

1.3.1.1. Достоинства

Исторически основным средством подготовки набора ТРС-Н является генератор dbgen, поставляемый вместе со спецификацией бенчмарка ТРС-Н [8; 11]. Его сильной стороной является полное следование эталонным правилам генерации и фактический статус исходной точки для проверки корректности других реализаций.

1.3.1.2. Ограничения

Вместе с тем dbgen отражает архитектурные предпосылки своего времени. Он ориентирован на формирование строковых файлов, не использует современные внутрипамятные колоночные представления, не обеспечивает прямой записи в современные форматы аналитического хранения и плохо приспособлен к сценариям, где требуется интеграция с удаленными объектными хранилищами. Дополнительно практическое использование dbgen ограничивается условиями лицензии ТРС, что затрудняет его прямую адаптацию для некоторых исследовательских и коммерческих сценариев.

1.3.2. Генератор tpchgen-rs

1.3.2.1. Достоинства

Одним из современных открытых аналогов является tpchgen-rs [13]. Это решение реализовано на языке Rust и ориентировано на параллельную генерацию таблиц и партиций. Существенным достоинством tpchgen-rs является поддержка прямого формирования данных в представлении Apache Arrow [1], что делает его существенно ближе к современной аналитической инфраструктуре, чем dbgen.

1.3.2.2. Ограничения

При этом tpchgen-rs в основном ориентирован на локальные сценарии использования и не решает в полной мере задачу прямой записи в удаленные S3-совместимые хранилища. Кроме того, для него не является центральной задачей архитектурного деления ядра генерации от слоя сериализации таким образом, чтобы генератор можно было использовать как универсальный встраиваемый источник данных для аналитических систем.

1.3.3. Сравнительный анализ и обоснование направления разработки

Сравнение рассмотренных решений показывает, что ни одно из них не сочетает одновременно все свойства, значимые для современного сценария эксплуатации анали-

тических систем: прямую генерацию в колоночное представление, запись в современные форматы хранения, работу с S3-совместимыми хранилищами, параллельное выполнение и возможность непосредственного встраивания в аналитические движки.

Именно эта совокупность требований определяет направление разработки Schengen [12]. Генератор должен сохранять детерминированность TPC-H и при этом быть ориентированным не на промежуточные строковые файлы, а на прямое формирование батчей Apache Arrow [1] с последующей сериализацией в актуальные форматы. Дополнительное расширение области применения дает интеграция генератора с DuckDB [6] в виде отдельного расширения, использующего прямую подачу данных в память СУБД.

Таблица 2 - Сравнение генераторов данных TPC-H

Критерий	dbgen	tpchgen-rs	Schengen
Основной формат вывода	tbl	прямая генерация, Arrow	прямая генерация, Arrow
Современные форматы хранения	требуется отдельная конвертация	частично поддерживаются	поддерживаются напрямую
Параллельная генерация	отсутствует	поддерживается	поддерживается
Работа с S3-совместимыми хранилищами	отсутствует	не является базовым сценарием	поддерживается напрямую
Встраивание в аналитические системы	практически отсутствует	ограничено	предусмотрено архитектурой
Лицензионная модель	ограничения TPC	открытая	открытая

Выводы по главе

В главе показано, что TPC-H остается актуальной основой для оценки аналитических СУБД, однако требования к генераторам данных в современной инфраструктуре существенно расширились. Помимо корректного воспроизведения эталонной схемы и распределений, генератор должен поддерживать колоночное внутрипамятное представление, прямую сериализацию в современные форматы, работу с удаленными хранилищами и встраивание в аналитические системы.

Анализ dbgen и tpchgen-rs показывает, что существующие решения закрывают эти требования лишь частично. Это обосновывает разработку Schengen как генератора, ориентированного на современные сценарии аналитической эксплуатации при сохранении детерминированности и совместимости с правилами TPC-H.

Глава 2. Архитектура генератора данных

Во второй главе рассматриваются архитектурные решения, за счет которых генератор масштабируется по числу партиций и рабочих потоков, но при этом сохраняет детерминированность результата. Основное внимание уделяется общей схеме работы программы, независимому запуску генераторов для произвольной партиции, архитектуре слоя сериализации, в которой параллельное построение батчей сочетается с линейным порядком записи, и разделению генераторного ядра и слоя сериализации.

2.1. Архитектурные требования к масштабируемой генерации

Базовое требование к архитектуре генератора состоит в сохранении совместимости со спецификацией TRC-H [8]. Это означает, что при одинаковых параметрах запуска должны воспроизводиться те же ключевые зависимости, распределения значений и итоговый логический порядок строк независимо от числа потоков и разбиения таблицы на партиции.

Второе требование связано с масштабируемостью. Параллельная генерация не должна опираться на последовательный прогон всех предшествующих строк, поскольку в таком случае стоимость подготовки очередной партиции росла бы пропорционально ее смещению относительно начала таблицы. Следовательно, архитектура должна поддерживать быстрый переход к произвольной строке без симуляции всего префикса.

Третье требование относится к записи результата. Слой сериализации должен оставаться отделенным от ядра генерации и принимать готовые батчи Apache Arrow [1]. При этом для режимов записи в один файл требуется обеспечить одновременно три свойства: параллельную генерацию партиций, линейный порядок данных в итоговом артефакте и детерминированность результата.

2.2. Общая идея алгоритма

Функционирование программы организовано как последовательность этапов выбора сценария генерации, формирования таблиц и записи результата.

На первом этапе программа обрабатывает параметры командной строки, определяющие масштаб генерации, состав таблиц, формат вывода и путь размещения результата. После этого выполняется регистрация метаданных таблиц и выбирается требуемый драйвер сериализации.

На втором этапе для каждой выбранной таблицы определяется объем данных и схема разбиения на логические партиции. Далее для каждой партиции запускается генератор, формирующий значения атрибутов по детерминированным правилам TRC-H.

На третьем этапе сгенерированные значения накапливаются и объединяются в батчи для увеличения пропускной способности. После этого батчи передаются слою

сериализации, который сохраняет результат в требуемом формате хранения.

2.3. Параллельная генерация на основе перемотки линейного генератора

2.3.1. Математическая модель генератора

В основе большинства генераторов значений ТРС-Н лежит линейный конгруэнтный генератор без аддитивного слагаемого. Для каждой последовательности используется рекуррентное соотношение

$$X_{n+1} = aX_n \bmod m, \quad (1)$$

где для 32-разрядного варианта ТРС-Н принимаются

$$a = 16807, \quad m = 2^{31} - 1. \quad (2)$$

Отдельные генераторы атрибутов используют собственные начальные значения. Начальное состояние конкретного столбца дополнительно смещается относительно базового значения, поэтому разные атрибуты не разделяют одну и ту же последовательность случайных чисел. Если генератор g расходует s_g значений на одну строку, то начало строки с номером r соответствует состоянию

$$X_{rs_g}^{(g)}. \quad (3)$$

Из этого следует, что для партии, начинающейся со строки r_0 , генератору не требуется воспроизводить весь префикс таблицы. Достаточно перейти к состоянию

$$X_{r_0s_g}^{(g)}, \quad (4)$$

после чего строки диапазона $[r_0, r_1)$ могут строиться независимо от других партий. Если внутри текущей строки уже было использовано u_g значений, то для перехода к следующей строке выполняется добор

$$\Delta_g = s_g - u_g, \quad (5)$$

что восстанавливает фиксированный шаг по строкам и сохраняет совместимость с эталонным порядком использования случайных чисел.

2.3.2. Алгоритм быстрой перемотки и асимптотика

Поскольку используемый генератор является мультипликативным, переход на k шагов вперед выражается в замкнутой форме:

$$X_{n+k} = a^k X_n \bmod m. \quad (6)$$

Тем самым задача перемотки сводится к вычислению $a^k \bmod m$.

```
func skip_ahead(state, steps, multiplier, modulus):
    factor = multiplier
    result = state
    remaining = steps

    while remaining > 0:
        if remaining mod 2 == 1:
            result = (result * factor) mod modulus

        factor = (factor * factor) mod modulus
        remaining = remaining // 2

    return result
```

Листинг 1 - Перемотка линейного генератора

Такой алгоритм выполняется за $O(\log k)$ при $O(1)$ дополнительной памяти, тогда как последовательный пропуск потребовал бы $O(k)$ шагов. Поэтому рабочий поток может сразу восстановить состояние генератора для начала своей партии и строить строки независимо от предшествующих партий. Именно это делает параллельную генерацию практически масштабируемой по числу партий.

2.4. Архитектура слоя сериализации

2.4.1. Форматно-зависимые компоненты записи

Слой сериализации проектируется как набор форматно-зависимых *writer*-компонентов, принимающих на вход готовые батчи Apache Arrow [1]. Для одних колоночных форматов используются файловые компоненты записи, поддерживающие как запись набора файлов-партий, так и запись в один итоговый файл. Для других колоночных форматов используются отдельные упорядоченные компоненты записи, передающие батчи во внешний рантайм формата.

Такое разделение позволяет сохранить единое ядро генерации и менять только последний этап конвейера. Генераторы таблиц всегда формируют один и тот же тип внутреннего результата, а различия между форматами и способами размещения данных локализованы внутри специализированных компонентов записи.

2.4.2. Линейный и детерминированный режим записи

В режиме раздельной записи партиций каждая партиция записывается собственным компонентом записи в отдельный файл. Такой режим хорошо масштабируется по числу рабочих потоков, поскольку между потоками отсутствует конкуренция за общий выходной артефакт.

Более сложной является архитектура режима упорядоченной записи в один итоговый файл. Здесь несколько производителей параллельно генерируют батчи партиций и передают их через ограниченную очередь Вьюкова MPSC (*multi-producer, single-consumer*) [14]. На стороне потребителя поддерживаются буфер готовых партиций и указатель ожидаемой партиции, задающий номер следующей партиции, которую разрешено зафиксировать в итоговом файле.

Если сообщение приходит от ожидаемой партиции, ее батчи сразу передаются в единый компонент записи. Если батч относится к более поздней партиции, он временно буферизуется до тех пор, пока не будут полностью записаны все предшествующие партиции. Тем самым итоговый файл всегда формируется в порядке номеров партиций, а не в порядке фактической готовности рабочих потоков.

Схема ручного упорядочения партиций для форматов с упорядоченной записью в один файл приведена на рисунке 1. На ней показано, как параллельные производители передают сообщения в общую очередь, а потребитель линейно фиксирует партиции в порядке их номеров.

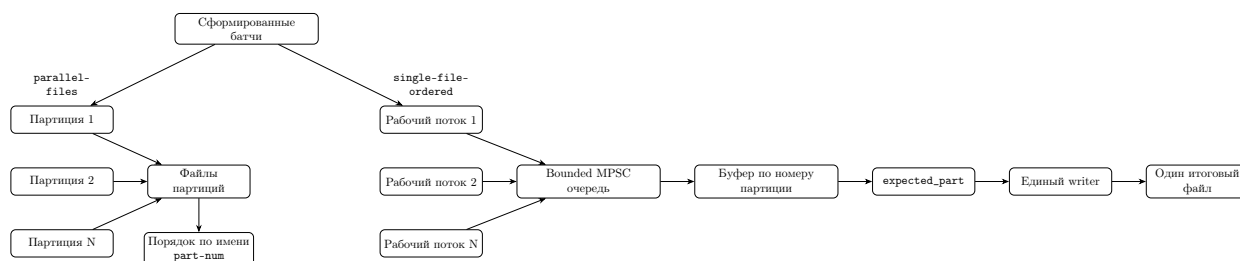


Рис. 1 - Схема ручного упорядочения партиций для форматов с записью в один файл

Линейность обеспечивается самим инвариантом ожидаемой партиции: никакая партиция не может быть записана раньше всех предшествующих. Детерминированность следует из того, что содержимое каждой партиции уже фиксировано на этапе генерации, а порядок записи определяется только номером партиции. Выбор очереди Вьюкова MPSC здесь важен как практический способ совместить высокий уровень параллелизма производителей с одним упорядочивающим потребителем.

2.5. Разделение модулей генерации и сериализации

Дополнительное архитектурное решение состоит в явном разделении модуля генерации и слоя сериализации. Генератор отвечает только за построение строк, партиций и батчей Apache Arrow, тогда как сериализация начинается только после формирования

очередного батча. Благодаря этому граница между компонентами проходит по устойчивому внутреннему представлению данных, а не по формату выходного файла.

Такое разделение оказалось важным не только для поддержки нескольких форматов хранения, но и для встраиваемого сценария. В расширении для DuckDB [6; 7] используется только генераторная часть конвейера: вместо передачи батчей в файловый компонент записи они сразу экспортируются во внешний табличный источник. Тем самым архитектурное отделение сериализации от генерации позволяет использовать один и тот же генератор и как автономную утилиту записи файлов, и как внутримаемный источник данных для аналитической СУБД.

Выводы по главе

Во второй главе выделены два архитектурных решения, определяющих масштабированность генератора. Первое из них состоит в быстрой перематке линейного генератора случайных чисел, благодаря которой партии могут обрабатываться независимо друг от друга. Второе связано с построением слоя сериализации как набора форматно-зависимых компонентов записи и использованием очереди MPSC для упорядоченной записи в один файл.

Тем самым на архитектурном уровне обеспечиваются параллельное масштабирование по числу партий, линейный детерминированный порядок результата при записи и возможность повторного использования только генераторной части конвейера во встраиваемом сценарии.

Глава 3. Реализация генератора данных

В третьей главе рассматривается практическая реализация генератора данных ТРС-Н [12]. Здесь основное внимание уделяется тому, как устроены формирование табличных данных, подготовка колоночных батчей, запись результата в разные форматы и использование генератора как встроенного источника данных.

3.1. Реализация генерации таблиц

3.1.1. Организация генерации по партициям

Реализация генерации начинается с выбора требуемых таблиц и подготовки общего текстового пула. После этого для каждой таблицы формируется набор логических партиций, а для каждой партиции создается собственный контекст генерации с диапазоном строк, масштабом генерации и параметрами размера батча. При запуске обработки партиции внутренние генераторы сразу переводятся к началу нужного диапазона строк, после чего партиция выдает последовательность колоночных батчей. Каждый батч сохраняет положение своего первого ряда в таблице, поэтому несколько рабочих потоков могут параллельно строить батчи разных партиций и передавать их в следующий слой без потери локального порядка строк.

3.1.2. Генерация значений и использование текстового пула

Для числовых атрибутов используются ограниченные генераторы целых значений. Они применяются для количеств, балансов, скидок, налогов, датовых смещений и других величин, для которых спецификацией ТРС-Н задан конечный диапазон допустимых значений. Для больших диапазонов ключей используется расширенный вариант генерации, допускающий работу с 64-разрядными значениями.

Для строковых атрибутов применяются несколько способов формирования значения. Значения из фиксированных словарей, например сегменты рынка, приоритеты заказов, типы деталей, инструкции отгрузки и режимы доставки, выбираются из взвешенных распределений. Имена деталей формируются как последовательности токенов, а адреса строятся как алфавитно-цифровые строки переменной длины. Телефонные номера вычисляются из ключа страны и случайных локальных компонент.

Генерация текстовых атрибутов в ТРС-Н задается набором распределений из файла `dists.dss` [8; 11]. Каждый блок содержит имя распределения и число элементов `COUNT`. Далее приводится список токенов с соответствующими весами; вес определяет относительную частоту выбора токена.

```
# <token> | <weight> # comment
```

```
BEGIN articles
```

```
COUNT|3
```

```

the|50
a|20
an|5
END articles

```

Листинг 2 - Общий вид распределения

Пусть распределение содержит токены t_i с весами w_i , а

$$W = \sum_{i=1}^n w_i. \quad (7)$$

Тогда выбирается случайное число $u \in [0, W)$, и возвращается первый токен t_j , для которого выполнено условие

$$\sum_{i=1}^j w_i > u. \quad (8)$$

Именно эта схема накопления весов используется при выборе значений из словарей.

Распределение **grammar** задает верхнеуровневую схему предложения. Символы N, V, P и T обозначают именную группу, глагольную группу, предложную группу и знак завершения предложения.

```

###
# grammar
# first level grammar. N=noun phrase, V=verb phrase,
# P=prepositional phrase, T=sentence termination
##
BEGIN grammar
COUNT|5
N V T|3
N V P T|3
N V N T|3
N P V N T|1
N P V P T|1
END grammar

```

Листинг 3 - Верхнеуровневая грамматика предложения

Далее нетерминальные символы раскрываются через распределения **np** и **vp**, которые определяют внутреннюю форму именной и глагольной групп.

```

###
# NP
BEGIN np
COUNT|4
N|10
J N|20
J, J N|10
D J N|50

```

```

END np

###
# VP
BEGIN vp
COUNT|4
V|30
X V|1
V D|40
X V D|1
END vp

```

Листинг 4 - Раскрытие именной и глагольной групп

Терминальные символы подставляются из словарей отдельных частей речи, например `nouns` и `verbs`.

```

BEGIN nouns
COUNT|45
packages|40
requests|40
accounts|40
deposits|40
foxes|20
ideas|20
END nouns

BEGIN verbs
COUNT|40
sleep|20
wake|20
are|20
cajole|20
haggle|20
END verbs

```

Листинг 5 - Словари терминальных токенов

Из получаемых предложений заранее формируется текстовый пул - общий текстовый буфер. При генерации конкретного комментария программа выбирает в нем смещение и длину фрагмента, после чего использует соответствующий срез как значение атрибута. Такой подход уменьшает накладные расходы при генерации длинных текстовых полей. Для некоторых таблиц, например `supplier`, поверх выбранного текста дополнительно может применяться модификация по правилу ВВВ¹, предусмотренному спецификацией ТРС-Н [8].

¹Правило ВВВ применяется при формировании атрибута `s_comment` таблицы `supplier`: в текст комментария вставляется подстрока «Customer Complaints» или «Customer Recommends» [8].

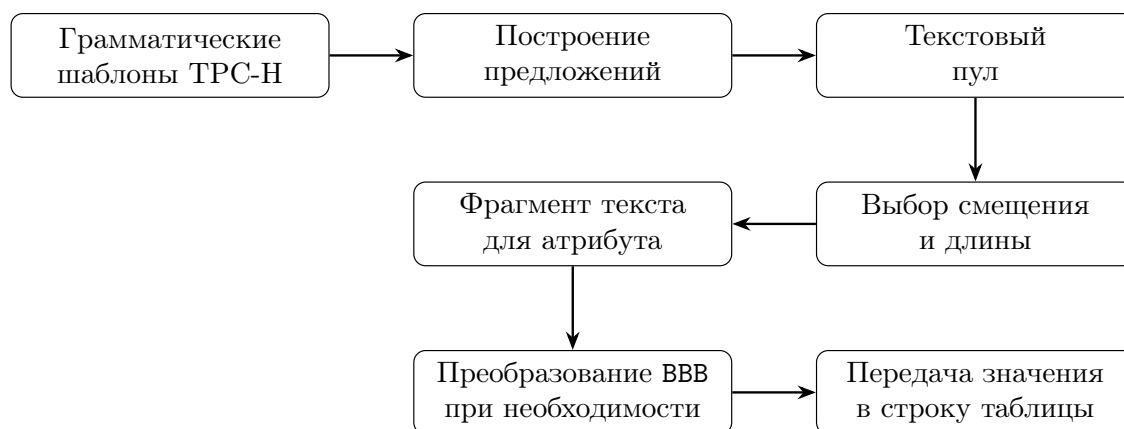


Рис. 2 - Схема формирования текстовых атрибутов на основе пула текста

3.1.3. Формирование таблиц набора данных

Для каждой таблицы используется собственный генератор, реализующий правила формирования ключей и зависимых атрибутов, заданные спецификацией ТРС-Н.

3.1.3.1. Таблица region

Атрибут	Ключ	Правило формирования
r_regionkey	РК	Последовательная нумерация строк, начиная с нуля.
r_name		Выбирается из фиксированного справочника регионов ТРС-Н.
r_comment		Выбирается из текстового пула.

Символическая запись генератора region приведена в приложении А.

3.1.3.2. Таблица nation

Атрибут	Ключ	Правило формирования
n_nationkey	РК	Последовательная нумерация строк, начиная с нуля.
n_name		Выбирается из предопределенного справочника наций.
n_regionkey	FK	Вычисляется по таблице наций и весам распределения регионов, заданным спецификацией ТРС-Н.
n_comment		Выбирается из текстового пула.

Символическая запись генератора nation приведена в приложении Б.

3.1.3.3. Таблица supplier

Атрибут	Ключ	Правило формирования
s_suppkey	РК	Номер строки с единичным смещением.
s_name		Формируется по шаблону Supplier#%09d.
s_address		Формируется как алфавитно-цифровая строка переменной длины.
s_nationkey	FK	Выбирается из допустимого диапазона ключей наций.
s_phone		Вычисляется из ключа страны и случайных локальных компонент.
s_acctbal		Генерируется как ограниченное числовое значение и переводится в денежный формат.
s_comment		Выбирается из текстового пула; при выполнении условия может модифицироваться по правилу ВВВ.

Символическая запись генератора supplier приведена в приложении В.

3.1.3.4. Таблица part

Атрибут	Ключ	Правило формирования
p_partkey	РК	Последовательная нумерация строк с единичным смещением.
p_name		Формируется как последовательность токенов из словаря цветов.
p_mfgr		Строится по номеру производителя.
p_brand		Вычисляется по номеру производителя и дополнительному номеру бренда.
p_type		Выбирается из словаря типов деталей.
p_size		Генерируется из ограниченного числового диапазона.
p_container		Выбирается из словаря контейнеров.
p_retailprice		Детерминированно вычисляется по ключу детали.
p_comment		Выбирается из текстового пула.

Символическая запись генератора part приведена в приложении Г.

3.1.3.5. Таблица partsupp

Атрибут	Ключ	Правило формирования
ps_partkey	FK	Соответствует текущей детали.
ps_suppkey	FK	Вычисляется как детерминированное отображение пары «ключ детали, номер поставщика»; для каждой детали формируются четыре поставщика.
ps_availqty		Генерируется из ограниченного диапазона доступного количества.
ps_supplycost		Генерируется из ограниченного диапазона и переводится в денежный формат.
ps_comment		Выбирается из текстового пула.

Символическая запись генератора partsupp приведена в приложении Д.

3.1.3.6. Таблица customer

Атрибут	Ключ	Правило формирования
c_custkey	PK	Последовательная нумерация строк с единичным смещением.
c_name		Формируется по шаблону Customer#%09d.
c_address		Формируется как алфавитно-цифровая строка переменной длины.
c_nationkey	FK	Выбирается из допустимого диапазона ключей наций.
c_phone		Вычисляется из ключа страны и случайных локальных компонент.
c_acctbal		Генерируется как ограниченное числовое значение и переводится в денежный формат.
c_mktsegment		Выбирается из словаря сегментов рынка.
c_comment		Выбирается из текстового пула.

Символическая запись генератора customer приведена в приложении Е.

3.1.3.7. Таблица orders

Атрибут	Ключ	Правило формирования
o_orderkey	РК	Строится по разреженной схеме из порядкового номера заказа.
o_custkey	FK	Выбирается из диапазона клиентов с исключением недопустимых ключей.
o_orderstatus		Определяется по числу отгруженных позиций заказа.
o_totalprice		Вычисляется как агрегат по позициям соответствующего заказа.
o_orderdate		Генерируется в допустимом диапазоне дат ТРС-Н.
o_orderpriority		Выбирается из словаря приоритетов заказов.
o_clerk		Формируется по шаблону Clerk#%09d.
o_shippriority		Имеет фиксированное значение 0.
o_comment		Выбирается из текстового пула.

Символическая запись генератора orders приведена в приложении Ж.

3.1.3.8. Таблица lineitem

Атрибут	Ключ	Правило формирования
l_orderkey	FK	Согласован с ключом соответствующего заказа.
l_partkey	FK	Выбирается из диапазона ключей деталей.
l_suppkey	FK	Вычисляется по паре «ключ детали, номер поставщика».
l_linenumber		Задаёт порядковый номер позиции внутри заказа.
l_quantity		Генерируется из ограниченного числового диапазона.
l_extendedprice		Вычисляется по цене детали и количеству.
l_discount		Генерируется как процент скидки из ограниченного диапазона.
l_tax		Генерируется как процент налога из ограниченного диапазона.
l_returnflag		Определяется по отношению даты получения к текущей календарной отметке TPC-H.
l_linestatus		Определяется по отношению даты отгрузки к текущей календарной отметке TPC-H.
l_shipdate		Вычисляется как смещение относительно даты заказа.
l_commitdate		Вычисляется как смещение относительно даты заказа.
l_receiptdate		Вычисляется как смещение относительно даты отгрузки.
l_shipinstruct		Выбирается из словаря инструкций по отгрузке.
l_shipmode		Выбирается из словаря режимов доставки.
l_comment		Выбирается из текстового пула.

Символические записи подготовки диапазона и строки lineitem приведены в приложении 3.

3.2. Реализация слоя сериализации

3.2.1. Общий интерфейс записи

На этапе реализации слой сериализации принимает готовые колоночные батчи Apache Arrow и передает их в драйвер выбранного формата хранения. Для Parquet и ORC используются файловые компоненты записи, для Lance и Vortex - мосты во внешний форматный рантайм. Для локального пути и удаленного URI используется одна и та же схема размещения результата, поэтому генератору не требуется различать конкретный тип хранилища.

3.2.2. Форматы с управляемым стримингом

Для части форматов с управляемым стримингом, прежде всего Parquet и ORC, генератор сам управляет тем, как батчи группируются в выходные артефакты. В этом случае поддерживаются два сценария: раздельная запись партиций в набор файлов и упорядоченная запись в один итоговый файл. Оба сценария реализуются внутри слоя сериализации и используют один и тот же поток готовых батчей.

```
func deliver_batches(table, batches, output_format, write_strategy):
    strategy = resolve_selected_write_strategy(output_format, write_strategy)

    if strategy == "partition-files":
        return write_partition_files(table, batches, output_format)

    if strategy == "ordered-single-output":
        return write_ordered_single_output(table, batches, output_format)

    return stream_to_native_runtime(table, batches, output_format)
```

Листинг 6 - Выбор режима записи

В режиме раздельной записи партиций каждая партиция записывается независимо, что естественно сочетается с параллельным формированием данных. В режиме упорядоченной записи в один итоговый файл батчи от нескольких рабочих потоков должны быть дополнительно упорядочены перед фиксацией результата. Для этого используется очередь MPSC (*multi-producer, single-consumer*), а на стороне потребителя поддерживается буфер готовых партиций.

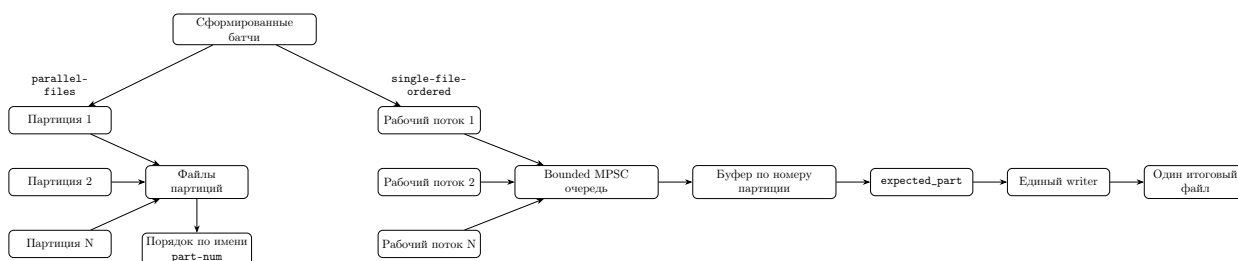


Рис. 3 - Схема ручного упорядочения партиций для форматов с управляемым стримингом

```
func write_ordered_single_output(table, batches, output_format):
    expected_part = 1
    buffered_parts = {}

    for message in batches:
        buffer_message(buffered_parts, message)
        flush_ready_partitions(buffered_parts, expected_part, output_format)
```

Листинг 7 - Упорядоченная запись в один итоговый файл

Такое решение позволяет совместить параллельную генерацию батчей с линейным порядком данных в итоговом файле. Именно этот механизм обеспечивает одинаковый логический порядок результата независимо от конфигурации рабочих потоков.

3.2.3. Форматы с нативным стримингом

Для форматов Lance [4] и Vortex [5] реализован другой сценарий: система последовательно передает уже сформированные батчи во внешний форматный рантайм. Тем самым генератор отвечает за границы партиций и порядок передачи данных, а физическое размещение и внутренняя структура набора данных определяются библиотекой соответствующего формата.

```
func stream_to_native_runtime(table, batches, output_format):
    runtime = open_runtime(output_format, table.schema)

    for partition in batches:
        runtime.begin_partition(partition.number)

        for batch in partition.batches:
            runtime.push_batch(batch)

    runtime.finish()
```

Листинг 8 - Передача батчей во внешний рантайм формата

3.3. Интерфейс запуска и сценарии эксплуатации

3.3.1. Параметры командной строки

Интерфейс запуска построен вокруг параметров командной строки. Пользователь может получить справку, вывести перечень таблиц, выбрать формат хранения, стратегию записи, каталог вывода и набор таблиц, которые необходимо сгенерировать. Фактический перечень доступных форматов зависит от активных драйверов текущей сборки.

Таблица 3 - Основные параметры командной строки

Параметр	Назначение
<code>-help</code>	Вывод справки по параметрам запуска.
<code>-list-tables</code>	Вывод списка поддерживаемых таблиц ТРС-Н.
<code>-list-formats</code>	Вывод форматов, доступных в текущей сборке.
<code>-scale-factor</code>	Задание масштаба генерации.
<code>-output-format</code>	Выбор целевого формата хранения.
<code>-output-path</code>	Путь или URI каталога вывода.
<code>-table</code>	Перечень таблиц для генерации; при отсутствии параметра формируются все таблицы.
<code>-batch-rows</code>	Максимальное число строк в одном батче.
<code>-write-strategy</code>	Выбор стратегии записи: <code>auto</code> , <code>parallel-files</code> , <code>single-file-ordered</code> .
<code>-write-workers</code>	Число рабочих потоков обработки партиций, 0 означает автоматический выбор.
<code>-write-queue-capacity</code>	Емкость очереди батчей при упорядоченной записи.
<code>-verbosity</code>	Режим вывода хода выполнения: <code>quiet</code> или <code>verbose</code> .

Помимо общих параметров, драйверы форматов добавляют собственные группы настроек: параметры сжатия, размеры буферов и ограничения на размер форматно-зависимых блоков хранения. В практическом использовании это позволяет не изменять общий интерфейс запуска при добавлении нового драйвера или новой стратегии записи.

3.3.2. Типовые сценарии запуска

Типовые сценарии эксплуатации приведены в листинге 9. Они охватывают получение служебной информации, генерацию нескольких таблиц, запись в единый файл и сохранение данных по URI.

```

schengen_main --help

schengen_main --list-formats

schengen_main \
  --scale-factor 10 \
  --output-format parquet \
  --output-path ./out \
  --table customer orders lineitem

schengen_main \
  --scale-factor 10 \
  --output-format orc \
  --output-path ./out_orc \
  --write-strategy single-file-ordered \
  --write-workers 4

```

```
schengen_main \  
  --scale-factor 10 \  
  --output-format lance \  
  --output-path s3://bucket/tpch \  
  --table lineitem
```

Листинг 9 - Примеры запуска генератора

Структура выходных данных определяется сочетанием формата и стратегии записи. При использовании раздельной записи партиций каталог таблицы содержит набор файлов-партиций. В режиме упорядоченной записи в один файл внутри каталога таблицы создается один итоговый артефакт. Такая организация позволяет единообразно адресовать результат по схеме `<output>/<table>/<artifact>`.

3.4. Реализация встраиваемого сценария

3.4.1. Идея встроенного сценария

Отделение генератора от слоя сериализации позволяет использовать его не только как автономную утилиту записи файлов, но и как источник табличных данных для аналитической СУБД. В таком сценарии сгенерированные в оперативной памяти батчи передаются непосредственно в движок OLAP-системы, минуя запись на диск, последующее чтение файлов, декодирование формата хранения и другие накладные расходы, характерные для файлового конвейера.

Практическая ценность этого варианта состоит в том, что генератор можно встраивать в систему исполнения запросов и использовать для моделирования нагрузки без предварительной материализации набора данных. Тем самым одна и та же реализация пригодна и для подготовки колоночных файлов, и для прямой подачи синтетических таблиц во внутренний движок СУБД.

3.4.2. Демонстрационный прототип для DuckDB

В качестве демонстрационного *proof-of-concept* такой сценарий реализован в виде отдельного расширения для DuckDB [6; 7]. Выбор именно этой СУБД обусловлен удобной инфраструктурой пользовательских расширений и сравнительно простой схемой подключения внешних табличных источников. Это позволяет проверить саму возможность интеграции генератора в аналитическую систему без существенной перестройки ядра генерации.

В расширении генераторная часть используется напрямую, без слоя сериализации. Пользователю доступны вызов `CALL schengen(scale_factor)` для публикации временных представлений и табличная функция `schengen_scan`, возвращающая одну выбранную таблицу. С точки зрения архитектуры этот прототип показывает, что гене-

ратор может работать как встроенный компонент СУБД и поставлять данные непосредственно в исполнитель запросов, не проходя через промежуточные форматы хранения.

```
LOAD schengen;  
CALL schengen(1);  
  
SELECT COUNT(*) FROM lineitem;  
SELECT * FROM schengen_scan('orders', 1) LIMIT 5;
```

Листинг 10 - Использование встроенного сценария через DuckDB

Выводы по главе

В третьей главе показано, как архитектурные решения предыдущей главы реализованы в программной системе. Генератор построен вокруг отдельных модулей формирования таблиц, общего текстового пула и колоночного представления Apache Arrow. Слой сериализации поддерживает несколько форматов хранения и две основные стратегии записи.

Также показано, что отделение ядра генерации от слоя сериализации позволило реализовать встраиваемый сценарий с DuckDB, в котором те же батчи используются без промежуточной записи на диск. Тем самым генератор поддерживает как автономную файловую эксплуатацию, так и внутривычислительную интеграцию с аналитической СУБД.

Заключение

Выполненная выпускная квалификационная работа посвящена разработке генератора данных для аналитического бенчмарка ТРС-Н, ориентированного на современную аналитическую инфраструктуру. В ходе работы проанализирована предметная область, рассмотрены существующие генераторы, сформулированы требования к масштабируемой архитектуре и реализована программная система, поддерживающая детерминированное формирование таблиц, подготовку колоночных батчей и запись результата в современные форматы хранения.

Поставленные задачи решены в полном объеме. Разработана архитектура, обеспечивающая параллельную генерацию на основе быстрой перемотки линейного генератора случайных чисел, реализован слой сериализации для нескольких колоночных форматов, обеспечена работа с локальными и удаленными хранилищами, а также показана возможность использования генератора без файлового слоя в качестве встроенного источника данных для аналитической СУБД.

Практическая значимость результата состоит в сокращении числа промежуточных этапов подготовки набора данных. Генератор позволяет сразу получать данные в колоночном представлении и целевых форматах хранения, что упрощает воспроизводимое проведение экспериментов и снижает накладные расходы на подготовку данных. Работоспособность встроенного сценария подтверждена реализацией отдельного расширения для DuckDB, выступающего как *proof-of-concept* использования генератора без слоя сериализации.

С точки зрения научно-технического уровня разработанное решение сочетает детерминированность эталонного ТРС-Н с прямой генерацией колоночных батчей, поддержкой современных форматов хранения и возможностью встроенного использования. Это позволяет рассматривать полученный результат как основу для дальнейшего развития генератора в направлении новых форматов и новых сценариев интеграции с аналитическими системами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Apache Arrow [Электронный ресурс] : A Cross-Language Development Platform for In-Memory Data. — 2026. — URL: <https://arrow.apache.org/> (дата обр. 07.04.2026).
2. Apache Parquet [Электронный ресурс] : Documentation. — 2026. — URL: <https://parquet.apache.org/docs/> (дата обр. 07.04.2026).
3. Apache ORC [Электронный ресурс] : Documentation. — 2026. — URL: <https://orc.apache.org/docs/> (дата обр. 07.04.2026).
4. Lance [Электронный ресурс] : Columnar Data Format and Dataset Format. — 2026. — URL: <https://github.com/lancedb/lance> (дата обр. 07.04.2026).
5. Vortex [Электронный ресурс] : An Adaptive File Format for Arrow. — 2026. — URL: <https://github.com/spiraldb/vortex> (дата обр. 07.04.2026).
6. Schengen-DuckDB [Электронный ресурс] : DuckDB extension for in-memory TPC-H data generation. — 2026. — URL: <https://github.com/m7kss1/schengen-duckdb> (дата обр. 07.04.2026).
7. Extensions [Электронный ресурс] / DuckDB Foundation. — 2026. — URL: <https://duckdb.org/docs/stable/extensions/overview.html> (дата обр. 07.04.2026).
8. TPC Benchmark H Standard Specification [Электронный ресурс] / Transaction Processing Performance Council. — Вер. 2.17.1. — 2024. — URL: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v2.17.1.pdf (дата обр. 07.04.2026).
9. Apache Iceberg [Электронный ресурс] : Documentation. — 2026. — URL: <https://iceberg.apache.org/> (дата обр. 07.04.2026).
10. *DeWitt D. J., Gray J.* Parallel Database Systems: The Future of High Performance Database Systems // Communications of the ACM. — 1992. — Т. 35, № 6. — С. 85–98.
11. tpch-dbgen [Электронный ресурс] : Mirror of the TPC-H dbgen implementation. — 2026. — URL: <https://github.com/electrum/tpch-dbgen> (дата обр. 07.04.2026).
12. Schengen [Электронный ресурс] : Data Generator for the TPC-H Analytical DBMS Benchmark. — 2026. — URL: <https://github.com/m7kss1/schengen> (дата обр. 07.04.2026).
13. tpchgen-rs [Электронный ресурс] : Parallel TPC-H data generator with Arrow support. — 2026. — URL: <https://lib.rs/crates/tpchgen> (дата обр. 07.04.2026).
14. *Vyukov D.* MPSC Queue [Электронный ресурс] / 1024cores. — 2026. — URL: <https://sites.google.com/site/1024cores/home/lock-free-algorithms/queues> (дата обр. 07.04.2026).

Генератор таблицы region

В приложении приведена символическая запись генератора таблицы region.

```
func generate_region(text_pool):  
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)  
  
    for region_key in range(0, REGION_COUNT):  
        emit RegionRow(  
            regionkey = region_key,  
            name = REGION_NAMES[region_key],  
            comment = comment_random.next_value()  
        )
```

Листинг А.1 - Формирование таблицы region

Генератор таблицы nation

В приложении приведена символическая запись генератора таблицы nation.

```
func generate_nation(text_pool):
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)

    for nation_key in range(0, NATION_COUNT):
        emit NationRow(
            nationkey = nation_key,
            name = NATION_NAMES[nation_key],
            regionkey = REGION_BY_NATION[nation_key],
            comment = comment_random.next_value()
        )
```

Листинг Б.1 - Формирование таблицы nation

Генератор таблицы supplier

В приложении приведена символическая запись генератора таблицы supplier.

```
func generate_supplier(partition_range, text_pool):
    address_random = RandomAlphaNumeric(address_seed, average_address_length)
    nation_random = RandomBoundedInt(nation_seed, 0, NATION_COUNT - 1)
    phone_random = RandomPhoneNumber(phone_seed)
    acctbal_random = RandomBoundedInt(acctbal_seed, acctbal_min, acctbal_max)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)
    bbb_gate_random = RandomBoundedInt(bbb_seed, 1, SCALE_BASE)
    bbb_junk_random = RowRandomInt(bbb_junk_seed, 1)
    bbb_offset_random = RowRandomInt(bbb_offset_seed, 1)
    bbb_type_random = RandomBoundedInt(bbb_type_seed, 0, 100)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        supplier_key = row_id + 1
        nation_key = nation_random.next_value()
        comment = copy(comment_random.next_value())

        if bbb_gate_random.next_value() <= BBB_COMMENTS_PER_SCALE_BASE:
            comment = apply_bbb_comment(comment, bbb_junk_random,
bbb_offset_random, bbb_type_random)

        emit SupplierRow(
            supkey = supplier_key,
            name = format("Supplier#%09d", supplier_key),
            address = address_random.next_value(),
            nationkey = nation_key,
            phone = phone_random.next_value(nation_key),
            acctbal = acctbal_random.next_value() / 100.0,
            comment = comment
        )

    finish_all_randoms_for_row()
```

Листинг В.1 - Формирование таблицы supplier

Генератор таблицы part

В приложении приведена символическая запись генератора таблицы part.

```
func generate_part(partition_range, text_pool):
    name_random = RandomStringSequence(name_seed, NAME_WORDS, color_distribution)
    manufacturer_random = RandomBoundedInt(mfgr_seed, mfgr_min, mfgr_max)
    brand_random = RandomBoundedInt(brand_seed, brand_min, brand_max)
    type_random = RandomString(type_seed, part_type_distribution)
    size_random = RandomBoundedInt(size_seed, size_min, size_max)
    container_random = RandomString(container_seed, container_distribution)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        part_key = row_id + 1
        mfgr_key = manufacturer_random.next_value()
        brand_key = mfgr_key * 10 + brand_random.next_value()

        emit PartRow(
            partkey = part_key,
            name = name_random.next_value(),
            mfgr = format("Manufacturer#%d", mfgr_key),
            brand = format("Brand#%d", brand_key),
            type = type_random.next_value(),
            size = size_random.next_value(),
            container = container_random.next_value(),
            retailprice = calculate_part_price(part_key) / 100.0,
            comment = comment_random.next_value()
        )

    finish_all_randoms_for_row()
```

Листинг Г.1 - Формирование таблицы part

Генератор таблицы partsupp

В приложении приведена символическая запись генератора таблицы partsupp.

```
func generate_partsupp(partition_range, scale_factor, text_pool):
    availqty_random = RandomBoundedInt(availqty_seed, availqty_min, availqty_max,
    4)
    supplycost_random = RandomBoundedInt(supplycost_seed, supplycost_min,
    supplycost_max, 4)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length,
    4)

    advance_all_randoms_to(partition_range.start_row)

    for part_row in range(partition_range.start_row, partition_range.end_row):
        part_key = part_row + 1

        for supplier_number in range(0, 4):
            supplier_key = select_part_supplier(part_key, supplier_number,
            scale_factor)

            emit PartSuppRow(
                partkey = part_key,
                suppkey = supplier_key,
                availqty = availqty_random.next_value(),
                supplycost = supplycost_random.next_value() / 100.0,
                comment = comment_random.next_value()
            )

        finish_all_randoms_for_row()
```

Листинг Д.1 - Формирование таблицы partsupp

Генератор таблицы customer

В приложении приведена символическая запись генератора таблицы customer.

```
func generate_customer(partition_range, text_pool):
    address_random = RandomAlphaNumeric(address_seed, average_address_length)
    nation_random = RandomBoundedInt(nation_seed, 0, NATION_COUNT - 1)
    phone_random = RandomPhoneNumber(phone_seed)
    acctbal_random = RandomBoundedInt(acctbal_seed, acctbal_min, acctbal_max)
    segment_random = RandomString(segment_seed, market_segment_distribution)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        customer_key = row_id + 1
        nation_key = nation_random.next_value()

        emit CustomerRow(
            custkey = customer_key,
            name = format("Customer#%09d", customer_key),
            address = address_random.next_value(),
            nationkey = nation_key,
            phone = phone_random.next_value(nation_key),
            acctbal = acctbal_random.next_value() / 100.0,
            mktsegment = segment_random.next_value(),
            comment = comment_random.next_value()
        )

    finish_all_randoms_for_row()
```

Листинг Е.1 - Формирование таблицы customer

Генератор таблицы orders

В приложении приведена символическая запись генератора таблицы orders.

```
func generate_orders(partition_range, scale_factor, text_pool):
    order_date_random = RandomBoundedInt(order_date_seed, order_date_min,
    order_date_max)
    line_count_random = RandomBoundedInt(line_count_seed, line_count_min,
    line_count_max)
    customer_random = next_bounded_value(customer_seed, 1, max_customer_key(
    scale_factor), scale_factor, 1)
    priority_random = RandomString(priority_seed, order_priority_distribution)
    clerk_random = RandomBoundedInt(clerk_seed, 1, max_clerk(scale_factor))
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)

    quantity_random = RandomBoundedInt(quantity_seed, quantity_min, quantity_max,
    LINE_COUNT_MAX)
    discount_random = RandomBoundedInt(discount_seed, discount_min, discount_max,
    LINE_COUNT_MAX)
    tax_random = RandomBoundedInt(tax_seed, tax_min, tax_max, LINE_COUNT_MAX)
    part_random = next_bounded_value(part_seed, 1, max_part_key(scale_factor),
    scale_factor, LINE_COUNT_MAX)
    ship_date_random = RandomBoundedInt(ship_date_seed, ship_date_min,
    ship_date_max, LINE_COUNT_MAX)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        order_index = row_id + 1
        order_key = make_sparse_order_key(order_index)
        order_date = order_date_random.next_value()
        customer_key = adjust_customer_key(customer_random.next_value(),
        max_customer_key(scale_factor))

        total_price = 0
        shipped_count = 0
        line_count = line_count_random.next_value()

        for line_number in range(1, line_count + 1):
            quantity = quantity_random.next_value()
            discount = discount_random.next_value()
            tax = tax_random.next_value()
            part_key = part_random.next_value()
            ship_date = order_date + ship_date_random.next_value()

            total_price += calculate_line_total(part_key, quantity, discount, tax)
```

```

        if is_in_past(ship_date):
            shipped_count += 1

    emit OrderRow(
        orderkey = order_key,
        custkey = customer_key,
        orderstatus = derive_order_status(shipped_count, line_count),
        totalprice = total_price / 100.0,
        orderdate = order_date,
        orderpriority = priority_random.next_value(),
        clerk = format("Clerk#%09d", clerk_random.next_value()),
        shippriority = 0,
        comment = comment_random.next_value()
    )

    finish_all_randoms_for_row()

```

Листинг Ж.1 - Формирование таблицы orders

Генератор таблицы lineitem

В приложении приведены символические записи подготовки диапазона заказов и формирования отдельной строки таблицы lineitem.

```
func generate_lineitem(order_partition, scale_factor, text_pool):
    order_date_random = RandomBoundedInt(order_date_seed, order_date_min,
    order_date_max)
    line_count_random = RandomBoundedInt(line_count_seed, line_count_min,
    line_count_max)
    quantity_random = RandomBoundedInt(quantity_seed, quantity_min, quantity_max,
    LINE_COUNT_MAX)
    discount_random = RandomBoundedInt(discount_seed, discount_min, discount_max,
    LINE_COUNT_MAX)
    tax_random = RandomBoundedInt(tax_seed, tax_min, tax_max, LINE_COUNT_MAX)
    part_random = next_bounded_value(part_seed, 1, max_part_key(scale_factor),
    scale_factor, LINE_COUNT_MAX)
    supplier_number_random = RandomBoundedInt(supplier_number_seed, 0, 3,
    LINE_COUNT_MAX)
    ship_date_random = RandomBoundedInt(ship_date_seed, ship_date_min,
    ship_date_max, LINE_COUNT_MAX)
    commit_date_random = RandomBoundedInt(commit_date_seed, commit_date_min,
    commit_date_max, LINE_COUNT_MAX)
    receipt_date_random = RandomBoundedInt(receipt_date_seed, receipt_date_min,
    receipt_date_max, LINE_COUNT_MAX)
    return_flag_random = RandomString(return_flag_seed, return_flag_distribution,
    LINE_COUNT_MAX)
    ship_instruct_random = RandomString(ship_instruct_seed,
    ship_instruct_distribution, LINE_COUNT_MAX)
    ship_mode_random = RandomString(ship_mode_seed, ship_mode_distribution,
    LINE_COUNT_MAX)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length,
    LINE_COUNT_MAX)
    generators = LineitemGenerators(
        quantity_random,
        discount_random,
        tax_random,
        part_random,
        supplier_number_random,
        ship_date_random,
        commit_date_random,
        receipt_date_random,
        return_flag_random,
        ship_instruct_random,
        ship_mode_random,
        comment_random
```

```

)

advance_all_randoms_to(order_partition.start_row)

for order_row in range(order_partition.start_row, order_partition.end_row):
    order_index = order_row + 1
    order_key = make_sparse_order_key(order_index)
    order_date = order_date_random.next_value()
    line_count = line_count_random.next_value()

    for line_number in range(1, line_count + 1):
        emit build_lineitem_row(
            order_key,
            line_number,
            order_date,
            scale_factor,
            generators
        )

finish_all_randoms_for_row()

```

Листинг 3.1 - Подготовка диапазона заказов для lineitem

```

func build_lineitem_row(order_key, line_number, order_date, scale_factor,
    generators):
    part_key = generators.part_random.next_value()
    supplier_number = generators.supplier_number_random.next_value()
    supplier_key = select_part_supplier(part_key, supplier_number, scale_factor)
    quantity = generators.quantity_random.next_value()
    discount = generators.discount_random.next_value()
    tax = generators.tax_random.next_value()

    ship_date = order_date + generators.ship_date_random.next_value()
    commit_date = order_date + generators.commit_date_random.next_value()
    receipt_date = ship_date + generators.receipt_date_random.next_value()

    return LineitemRow(
        orderkey = order_key,
        partkey = part_key,
        suppkey = supplier_key,
        linenumber = line_number,
        quantity = quantity,
        extendedprice = calculate_part_price(part_key) * quantity / 100.0,
        discount = discount / 100.0,
        tax = tax / 100.0,
        returnflag = derive_return_flag(receipt_date, generators.
return_flag_random),
        linestatus = derive_line_status(ship_date),
        shipdate = ship_date,

```

```
        commitdate = commit_date,  
        receiptdate = receipt_date,  
        shipinstruct = generators.ship_instruct_random.next_value(),  
        shipmode = generators.ship_mode_random.next_value(),  
        comment = generators.comment_random.next_value()  
    )
```

Листинг 3.2 - Формирование строки lineitem